



Projekt

PORR 25Z

Operacja splotu na obrazie z wykorzystaniem filtra do wykrywania krawędzi

Jakub Bliźniuk
Mateusz Szyperek
Kalina Białek

Prowadzący: dr. Mateusz Koryciński

14 January 2026

Spis treści

1 Opis projektu	1
2 Opis tematu projektu	1
3 Implementacje tematu projektu	3
3.1 Wersja sekwencyjna	3
3.2 OpenMP	4
3.3 MPI	4
3.4 GPU (SYCL)	6
4 Benchmarki	8
5 Podsumowanie i wnioski	13
Bibliografia	13
Spis załączników	13
Załącznik A Pełne wyniki benchmarków	14

1 Opis projektu

Projekt koncentruje się na zaimplementowaniu rozwiązania wybranego tematu w czterech wersjach: sekwencyjnej oraz zrównoleglonej w trzech różnych technologiach. Następnie wykonuje się porównanie czasu obliczeń dla przykładowych instancji problemu i przeprowadza analizę wyników.

2 Opis tematu projektu

Tematem projektu jest **operacja splotu na obrazie z wykorzystaniem filtra do wykrywania krawędzi**. Ze znanych powszechnie operatorów został wybrany Sobel, który bazuje na idei Irwina Sobela oraz Gary’ego M. Feldmana przedstawionej w referacie z 1968 roku. Do wykrywania krawędzi używa się operatora dyskretnego różniczkowania i wykorzystuje się go do aproksymacji gradientu intensywności obrazu. Dzięki temu jesteśmy w stanie zlokalizować krawędzie, interpretując je jako nagłe zmiany w intensywności [1], [2].

Operator używa dwóch jąder przekształcenia w formie macierzy 3×3 o wartościach całkowitych – jednego do aproksymacji gradientu horyzontalnych zmian w intensywności obrazu, a drugiego do aproksymacji gradientu zmian w kierunku wertykalnym. Obliczenia są wykonywane oddzielnie dla każdego piksela, a więc istnieje potencjał do ich zrównoleglenia.

Pierwszym krokiem jest konwersja obrazu do odcieni szarości, co wyznacza nam wartości intensywności dla pikseli. Następnie dla każdego z pikseli wyodrębniamy fragment obrazu w formie macierzy pikseli 3×3 , w którym dany piksel jest środkowym elementem. Jeśli owy fragment wykraczałby poza granice obrazu, to „wystające” elementy są uzupełniane wartościami zerowymi. Nazwijmy ową macierz A .

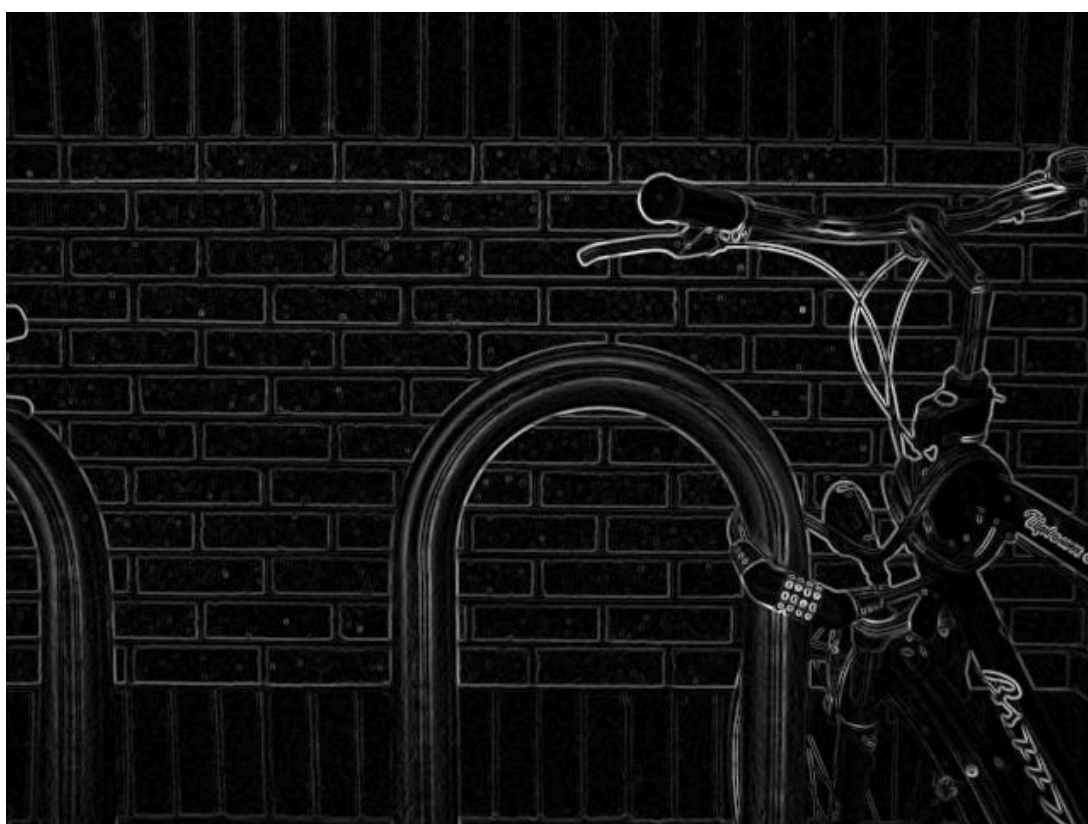
Niech $*$: $\mathbb{Z}^{3 \times 3} \times \mathbb{Z}^{3 \times 3} \rightarrow \mathbb{Z}$ to iloczyn po odpowiadających sobie elementach macierzy, a następnie suma wartości wszystkich jej elementów. Wykonujemy kolejno następujące obliczenia:

$$\begin{aligned} G_x &= \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * A \\ G_y &= \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} * A \\ G &= \sqrt{G_x^2 + G_y^2} \end{aligned} \tag{1}$$

W ten sposób otrzymujemy nowy obraz o wymiarach pierwotnego, który ma wyższe wartości pikseli na wykrytych krawędziach.



Rysunek 1. obraz w odcieniach szarości[2]



Rysunek 2. wynik użycia operatora Sobela[2]

3 Implementacje tematu projektu

Kod źródłowy znajduje się na platformie GitHub pod linkiem: [GitHub](#).

Implementacje rozdzielone są na wiele niezależnych od siebie gałęzi¹. Implementacja wersji sekwencyjnej znajduje się na gałęzi main, a implementacje zrównoleglone znajdują się na odpowiednio nazwanych gałęziach: SYCL dla GPU (przez framework SYCL), OpenMP i MPI. Implementacje działają niezależnie od siebie, więc w celu ich uruchomienia należy sklonować odpowiednią gałąź i postępować zgodnie z README.md dla wybranej implementacji.

Poza tym na branchu main znajduje się kod benchmarku, który wykonuje testy dla sumarycznie 13 wersji implementacji na CPU i GPU.

3.1 Wersja sekwencyjna

Wersja sekwencyjna została zaimplementowana w języku C++. Cały kod znajduje się w pliku main.cpp.

- ImageF – obrazy są przechowywane w formie wektora liczb zmiennoprzecinkowych: vector<float>, do którego elementów odwołujemy się indeksem wyliczanym przez funkcję idx
- ImageF toGray(...) – funkcja konwertująca podany obraz do obrazu w odcieniach szarości
- ImageF convolve(...) – funkcja nakłada operator Sobela na każdy piksel obrazu po kolei (sekwencyjnie) i zwraca obraz złożony z wyników operacji
- vector<uint8_t> normalizeToU8(...) – funkcja normalizuje obraz typu ImageF do tablicy wartości typu uint8_t
- int main(...) – wczytuje obraz, przeprowadza na nim operacje zgodnie z opisem operatora Sobela, a następnie zapisuje otrzymany wynik w pliku

Nasza naiwna sekwencyjna konwolucja używająca operatora Sobela wygląda następująco:

Program 1. Sekwencyjna implementacji konwolucji

```
1  ImageF convolve(const ImageF &in, int w, int h, const std::vector<float>& kernel, int kw, int kh){  
2      ImageF out(w*h, 0.0f);  
3      int padX = kw/2, padY = kh/2;  
4      for(int y=0; y<h; ++y){  
5          for(int x=0; x<w; ++x){  
6              float s = 0.0f;  
7              for(int ky=0; ky<kh; ++ky){  
8                  for(int kx=0; kx<kw; ++kx){  
9                      int ix = x + kx - padX;  
10                     int iy = y + ky - padY;  
11                     if(ix>=0 && ix<w && iy>=0 && iy<h){  
12                         s += in[idx(ix,iy,w)] * kernel[ky*kw + kx];  
13                     }  
14                 }  
15             }  
16             out[idx(x,y,w)] = s;  
17         }  
    }
```

¹ Dodatkowo na gałęzi report znajduje się źródło tego raportu

```

18     }
19     return out;
20 }

```

3.2 OpenMP

OpenMP to API – wykorzystujące głównie dyrektywy kompilatora który rozszerza – do tworzenia programów dla systemów wielordzeniowych z dzieloną pamięcią. Pozwala ono na bardzo proste dodanie obsługi wielu wątków do programu i w naszym wypadku wymagało minimalnych modyfikacji w porównaniu do wersji sekwencyjnej.

Przykładowo w funkcji konwolucji główną zmianą – poza dodaniem dyrektywy `#pragma omp parallel for` – jest połączenie pętli po koordynatach `x` i `y` w jedną pętlę zrównoleżoną właśnie przez OpenMP.

Program 2. Implementacja konwolucji z użyciem OpenMP

```

1  ImageF convolve(const ImageF &in, int w, int h,
2                  const std::vector<float> &kernel, int kw, int kh) {
3      ImageF out(w * h, 0.0f);
4      int padX = kw / 2, padY = kh / 2;
5
6      #pragma omp parallel for schedule(static) default(none) shared(in, out, w, h, kernel, kw, kh,
7                               padX, padY)
8          for (int i = 0; i < w * h; ++i) {
9              int x = i % w;
10             int y = i / w;
11             float sum = 0.0f;
12             for (int ky = 0; ky < kh; ++ky) {
13                 for (int kx = 0; kx < kw; ++kx) {
14                     int ix = x + kx - padX;
15                     int iy = y + ky - padY;
16                     if (ix >= 0 && ix < w && iy >= 0 && iy < h) {
17                         sum += in[idx(ix, iy, w)] * kernel[ky * kw + kx];
18                     }
19                 }
20             }
21             out[i] = sum;
22         }
23     return out;
24 }

```

Mimo tego, przez możliwość zrównoleżenia, nastąpiła niewielka zmiana w implementacji pełnego algorytmu - wydzieliliśmy funkcję `magnitude` do obliczania wielkości gradientu, umożliwiając zrównoleżenie jej wewnętrznej pętli.

3.3 MPI

Message Passing Interface (MPI) to standard definiujący API do przekazywania wiadomości między procesami tworzony przez MPI Forum. W praktyce jest to więc rozwiązanie niższego poziomu w po-

równaniu do np. OpenMP, wymagające manualnego uruchamiania procesów bez prostych abstrakcji pozwalających np. na zrównoleglenie pętli przez dodanie jednej dyrektywy `#pragma parallel for`.

Model działania MPI jest także znacząco inny niż OpenMP, nie polegając na współdzielonej pamięci, co umożliwia rozdzielenie pracy na wiele osobnych systemów. Niestety nie odbywa się to bez kosztów i poza zwiększonym skomplikowaniem API, przekazywanie wiadomości ma większy narzut wydajnościowy w porównaniu do dostępu do tej samej pamięci.

Konsekwentnie, możemy zauważyć bardziej znaczące różnice w konstrukcji programu korzystającego z MPI w porównaniu do wersji sekwencyjnej i korzystającej z OpenMP. W naszym wypadku same wykonywane kroki algorytmu nie różnią się znacząco, natomiast nastąpiła duża zmiana w sposobie ich wywoływania: wersja w MPI dzieli obraz na wiele części z użyciem metody `MPI_Scatterv` i synchronizuje je między procesami – każdy proces dostaje więc swój mały obraz na którym może pracować, a na końcu są one łączone w jedną całość z użyciem `Gatherv`.

Dodatkowo procesy w obliczeniach korzystają z wierszy obrazu, które należą do innych procesów, co skutkuje potrzebą wysłania odpowiednio pierwszego i ostatniego wiersza danego fragmentu do „sąsiednich” procesów. Obsługa przekazywania sobie informacji między procesami powoduje kolejne opóźnienia.

Choć koncepcyjnie tego typu podejście jest dość proste – i ma zaletę względem choćby OpenMP w formie pełnej kontroli nad tym jaki proces ma dostęp do jakich danych – możemy zauważyć, że wymaga ono znacząco więcej kodu.

Program 3. Rozdzielenie obrazu między procesy z użyciem `MPI_Scatterv`

```
1 // Broadcast metadata
2 MPI_Bcast(&width, 1, MPI_INT, 0, MPI_COMM_WORLD);
3 MPI_Bcast(&height, 1, MPI_INT, 0, MPI_COMM_WORLD);
4 MPI_Bcast(&channels, 1, MPI_INT, 0, MPI_COMM_WORLD);
5
6 // Distribute work across processes
7 int rows_per_process = height / num_processes;
8 int remainder = height % num_processes;
9 int start_row = rank * rows_per_process + std::min(rank, remainder);
10 int end_row = start_row + rows_per_process + (rank < remainder ? 1 : 0);
11 int local_height = end_row - start_row;
12
13 // Send loaded image to other processes so they don't need to load them themselves
14 std::vector<int> sendcounts;
15 std::vector<int> displs;
16
17 if (rank == 0) {
18     sendcounts.resize(num_processes);
19     displs.resize(num_processes);
20
21     int current_disp = 0;
22     for (int r = 0; r < num_processes; r++) {
23         int r_start = r * rows_per_process + std::min(r, remainder);
24         int r_end = r_start + rows_per_process + (r < remainder ? 1 : 0);
25         int r_h = r_end - r_start;
```

```

26
27     sendcounts[r] = r_h * width * channels; // Note: Bytes, not pixels!
28     displs[r] = current_disp;
29     current_disp += sendcounts[r];
30 }
31 }
32
33 // Receive the image
34 std::vector<uint8_t> local_data(width * local_height * channels);
35
36 MPI_Scatterv(
37     data, sendcounts.data(), displs.data(), MPI_UNSIGNED_CHAR, // Send params
38     local_data.data(), local_height * width * channels, MPI_UNSIGNED_CHAR, // Recv params
39     0, MPI_COMM_WORLD // Root and Comm
40 );

```

Program 4. Zbieranie wyników do jednego obrazu

```

1 // Gather results cpp
2 ImageF final_result;
3 if (rank == 0) final_result.resize(width * height);
4
5 std::vector<int> recvcounts(num_processes);
6 std::vector<int> displs_v2(num_processes);
7 if (rank == 0) {
8     int current_disp = 0;
9     for (int r = 0; r < num_processes; r++) {
10         // Re-calculate the layout logic for other ranks
11         int r_start = r * rows_per_process + std::min(r, remainder);
12         int r_end = r_start + rows_per_process + (r < remainder ? 1 : 0);
13         int r_height = r_end - r_start;
14
15         recvcounts[r] = r_height * width;
16         displs_v2[r] = current_disp;
17         current_disp += recvcounts[r];
18     }
19 }
20
21 MPI_Gatherv(output.data(), local_height * width, MPI_FLOAT, final_result.data(), recvcounts.data(),
    displs_v2.data(), MPI_FLOAT, 0, MPI_COMM_WORLD);

```

3.4 GPU (SYCL)

Do zaimplementowania operatora Sobela na karcie graficznej wykorzystaliśmy SYCL – osadzony w C++ język do programowania heterogennego, skupiony na programowaniu GPU (choć z założenia ma wspierać też inne akceleratory, np. FPGA), utrzymywany przez grupę Khronos. Konceptyjnie jest inspirowany modelem CUDA i HIP, zapewniając podobne doświadczenie developerskie pisania kodu dzielonego z C++ na hoście, używając abstrakcji na wyższym poziomie niż Vulkan czy OpenCL. Cała implementacja znajduje się w pliku `main.cpp` – poza benchmarkiem, który został wydzielony do pliku `bench.cpp` i zawiera dodatkowe implementacje testowane w czasie prac nad ostateczną wersją.

SYCL opiera się na koncepcie projektowania kerneli na urządzenia (reprezentowanych jako funkcje — typowo lambdy) umieszczanych w specyficznych dla urządzenia kolejkach. Wykorzystaliśmy to, dzieląc program na trzy osobne kernele wykonywane w kolejności: przekształcenie obrazu na skalę szarości, filtr Sobela i normalizację wyników.

Typowo kernele implementowane są jako anonimowe wyrażenia lambda, do których przenieśliśmy naszą implementację zostawiając jako osobną nazwaną funkcję tylko `main`. Przykładowo, fragment kodu odpowiedzialny za konwolucję w wersji SYCL wygląda następująco:

Program 5. Implementacja konwolucji w SYCL

```
1  #pragma region Sobel cpp
2  // Sobel optimized for RDNA3: 128x1 work-groups, 2 pixels per work-item
3  // Might need adjustment for other GPUs, especially other vendors
4  int workW = (w + 1) / PIXELS_PER_WI;
5  int globalX = ((workW + WG_X - 1) / WG_X) * WG_X;
6  int globalY = h;
7
8  queue.parallel_for<sobel_kernel>({
9      sycl::nd_range<2>{
10         sycl::range<2>(globalY, globalX),
11         sycl::range<2>(WG_Y, WG_X)
12     },
13     [=](sycl::nd_item<2> item) {
14         int y = item.get_global_id(0);
15         int x0 = item.get_global_id(1) * 2;
16
17         if (y >= h) return;
18
19         // Process two adjacent pixels
20         #pragma unroll
21         for (int dx = 0; dx < PIXELS_PER_WI; ++dx) {
22             int x = x0 + dx;
23             if (x >= w) continue;
24
25             if (x > 0 && y > 0 && x < w - 1 && y < h - 1) {
26                 float p00 = gray[(y-1) * w + (x-1)];
27                 float p01 = gray[(y-1) * w + x];
28                 float p02 = gray[(y-1) * w + (x+1)];
29                 float p10 = gray[y * w + (x-1)];
30                 float p12 = gray[y * w + (x+1)];
31                 float p20 = gray[(y+1) * w + (x-1)];
32                 float p21 = gray[(y+1) * w + x];
33                 float p22 = gray[(y+1) * w + (x+1)];
34
35                 float gx = -p00 + p02 - 2.0f * p10 + 2.0f * p12 - p20 + p22;
36                 float gy = -p00 - 2.0f * p01 - p02 + p20 + 2.0f * p21 + p22;
37
38                 mag[y * w + x] = sycl::native::sqrt(gx * gx + gy * gy);
39             } else {
40                 mag[y * w + x] = 0.0f;
41             }
42         }
43     }
44 }
```

```

42     }
43 }
44 );
45 #pragma endregion

```

Poza realizacją głównej pętli jako dodanie mapującego kernela do kolejki przez `queue.parallel_for`, możemy na Programie 5 zauważyć znaczącą różnicę w implementacji: częściowo jest to unrolling pętli, co może pozytywnie wpłynąć także na wersję na CPU², ale co ważniejsze zmiana ta pozwala każdemu z naszych rdzeni GPU pracować nad tylko małą częścią problemu (2 pikselami³) z użyciem relatywnie prostych operacji. Pozwala to znacznie lepiej wykorzystać maszyną liczbę rdzeni dostępną na procesorze graficznym niż gdybyśmy portowali wersję na CPU w bardziej naiwny sposób.

Prawdopodobnie nie jest to też optymalna implementacja⁴ – bardziej zaawansowanym podejściem do optymalizacji na GPU jest „kafelkowanie”, gdzie grupuje się rdzenie w „kafelki” które najpierw ładują fragmenty całego zadania (tutaj nawet obrazu) do lokalnej dla grupy pamięci, a później pracują na niej bez konieczności wolnych dostępu do globalnych dla kernela danych.

Dwie główne implementacje SYCL to oneAPI od Intel’a i otwartoźródłowe AdaptiveCpp. W teorii oba kompilatory wspierają GPU zarówno Intel’a, jak i AMD oraz Nvidii, ale ze względu na problemy ze środowiskiem oneAPI zdecydowaliśmy się używać AdaptiveCpp.

4 Benchmarki

W gałęzi `main` znajduje się także program do testowania wydajności implementacji wykorzystując bibliotekę Benchmark od Google[3].

Jako platformę testową wykorzystaliśmy komputer klasy desktop z procesorem AMD Ryzen™ 9 7900X i procesorem graficznym AMD Radeon RX 7800 XT. Testy zostały wykonane na systemie NixOS 25.11 z kerneliem Linux w wersji 6.17.

Testowane były wersje AdaptiveCpp (SYCL) 25.02.0, LLVM/OpenMP 21.1.7 i MPI 5.0.9 zainstalowane z repozytorium NixPkgs.

- (v1) bazowa implementacja wersji sekwencyjnej
- (v2) implementacja wersji sekwencyjnej po zoptymalizowaniu wewnętrznej pętli
- (v3) zrównoleglona implementacja bazowej wersji sekwencyjnej z użyciem OpenMP
- (v4) zrównoleglona implementacja bazowej wersji sekwencyjnej z użyciem MPI
- (v5) zrównoleglona implementacja bazowej wersji sekwencyjnej z użyciem SYCL z użyciem buforów (starsza abstrakcja dzielenia pamięci między CPU/GPU)⁵

²dla kompletności wersja sekwencyjna konwolucji z tą implementacją została dodana do benchmarku

³Wartość 2 pikseli została znaleziona eksperymentalnie i jest specyficzna do używanego sprzętu. Bardzo możliwe, że inne architektury GPU – w szczególności Nvidii – lepiej sprawdzą się z inną konfiguracją rozmiaru grup i ilości pracy na grupę.

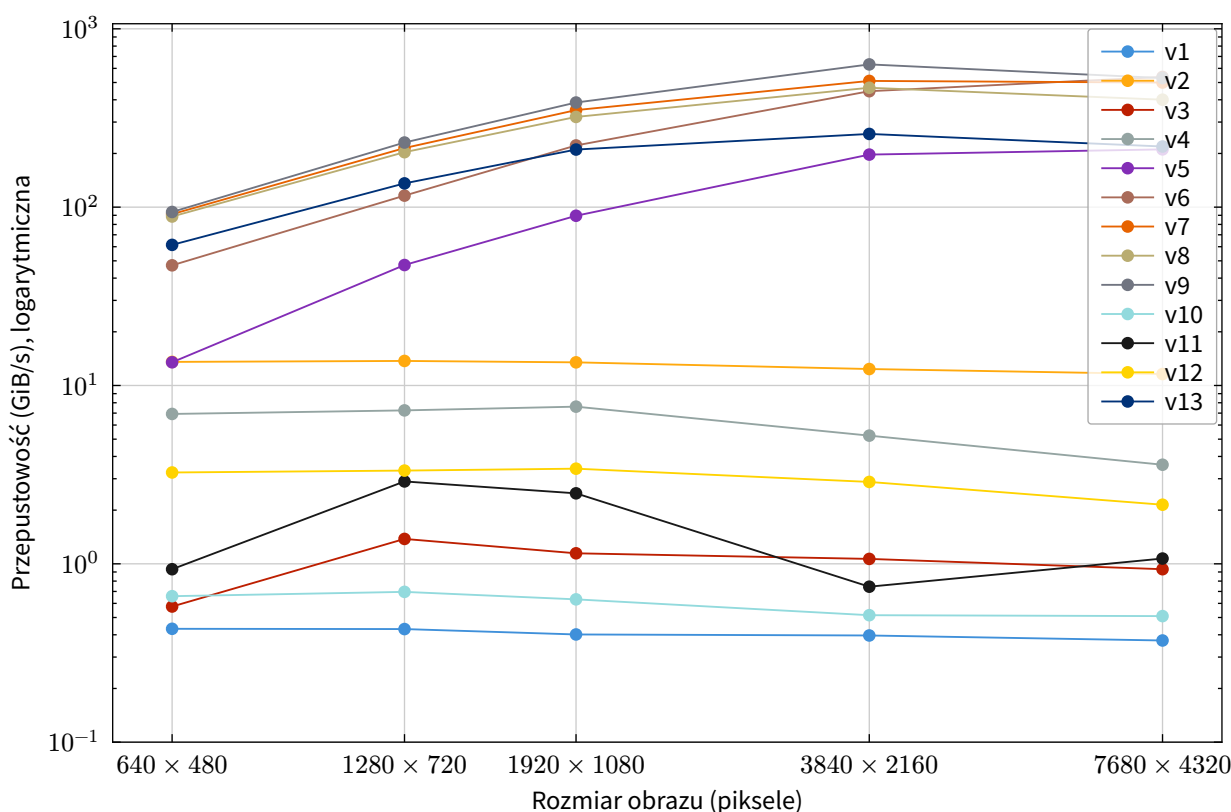
⁴Warto też wspomnieć, że prawdopodobnie możliwe by było także wykorzystanie wbudowanych w kartę graficzną prymitywów i potencjalnie hardware’u dedykowanego do pracy z obrazami. W ramach pracy zignorowaliśmy bowiem graficzną naturę zadania, przyjmując za cel zbadanie ogólnego przypadku.

⁵Obecnie bufory nie są rekomendowane przez gorszą wydajność — kompilator nawet ostrzega przed ich użyciem

- (v6) równoległa implementacja zoptymalizowanej wersji sekwencyjnej z użyciem SYCL z użyciem buforów (starsza abstrakcja dzielenia pamięci między CPU/GPU)
- (v7) równoległa implementacja zoptymalizowanej wersji sekwencyjnej z użyciem SYCL z użyciem USM (nowsza abstrakcja dzielenia pamięci między CPU/GPU)
- (v8) równoległa implementacja zoptymalizowanej wersji sekwencyjnej z użyciem SYCL i korzystająca z tilingu⁶
- (v9) zoptymalizowana równoległa implementacja z użyciem na USM, przetwarzająca po dwa piksele na wątek; uproszczona
- (v10) pełne przetwarzanie obrazu sekwencyjnie, wliczając przetwarzanie pikseli na skalę szarości i normalizację wyniku
- (v11) pełne przetwarzanie obrazu równoległe z wykorzystaniem OpenMP
- (v12) pełne przetwarzanie obrazu równoległe z wykorzystaniem MPI
- (v13) pełne przetwarzanie obrazu równoległe na GPU z użyciem kernela z (v9)

Dla każdego wariantu wykonano benchmark dla pięciu różnych rozdzielczości obrazów:

- 640×480
- 1280×720
- 1920×1080
- 3840×2160
- 7680×4320



Rysunek 3. Wszystkie wyniki benchmarków

⁶Podział zadania na grupy wątków (kafelki), gdzie każda grupa najpierw ładuje swoje dane do lokalnej pamięci, czeka na synchronizację, a później wykonuje operacje na lokalnych danych

Możemy zauważyć, że na samym dole wykresu znalazła się podstawowa wersja sekwencyjna i do szczytu jest na tyle duża, że konieczne było użycie skali logarytmicznej by różnice między najgorszymi implementacjami były widoczne.

Inaczej jednak przedstawia się sprawa po optymalizacji pętli (v2) - co prawda nadal przegrywa z większością wariantów na GPU, ale **wygrywa** ze zrównoleglonymi na procesorze naiwnymi wersjami.

Ale dlaczego implementacja na procesorze graficznym może osiągać tak znacząco lepsze wyniki niż cokolwiek na procesorze? Jest to konsekwencja adekwatności programu do alternatywnej, znacznie bardziej równoległej architektury GPU.

W przeciwieństwie do procesorów, które zawierają kilka do kilkudziesięciu (obecnie do 192 w niektórych procesorach serwerowych, choć w najbliższym czasie oczekiwane jest pojawienie się na rynku procesorów z nawet 256 rdzeniami) rdzeni, każdy z dużą pamięcią podręczną i zaawansowaną logiką do przyspieszania operacji sekwencyjnych, procesory graficzne wykorzystują bardzo dużą liczbę małych rdzeni, liczoną w tysiącach nawet po stronie konsumenckiej. Pojedynczy rdzeń jest jednak bardziej ograniczony niż w przypadku CPU i znacznie bardziej polega się na współdzieleniu zasobów (szczególnie pamięci) między rdzeniami, grupując je zwykle nawet w najmniejszym bloku w 32 lub 64 jednostki (warp w terminologii Nvidii i wavefront w terminologii AMD).

Ta liczba rdzeni oznacza, że zadania, które łatwo jest podzielić na bardzo wiele wątków — tak jak większość operacji na grafice — możemy przyspieszyć nawet setki razy względem CPU.

Dlaczego więc, poza oczywistymi ograniczeniami skalowania przy wzroście liczby wątków dla naszych problemów, nie wszystkie implementacje działają znacząco szybciej i dlaczego nie wykorzystujemy GPU do wszystkiego? W przypadku programowania GPU częstym ograniczeniem wydajności są czasy dostępu do różnych poziomów pamięci. W środowisku heterogennym dochodzi potrzeba zaprojektowania sposobu przeniesienia danych z procesora na akcelerator.

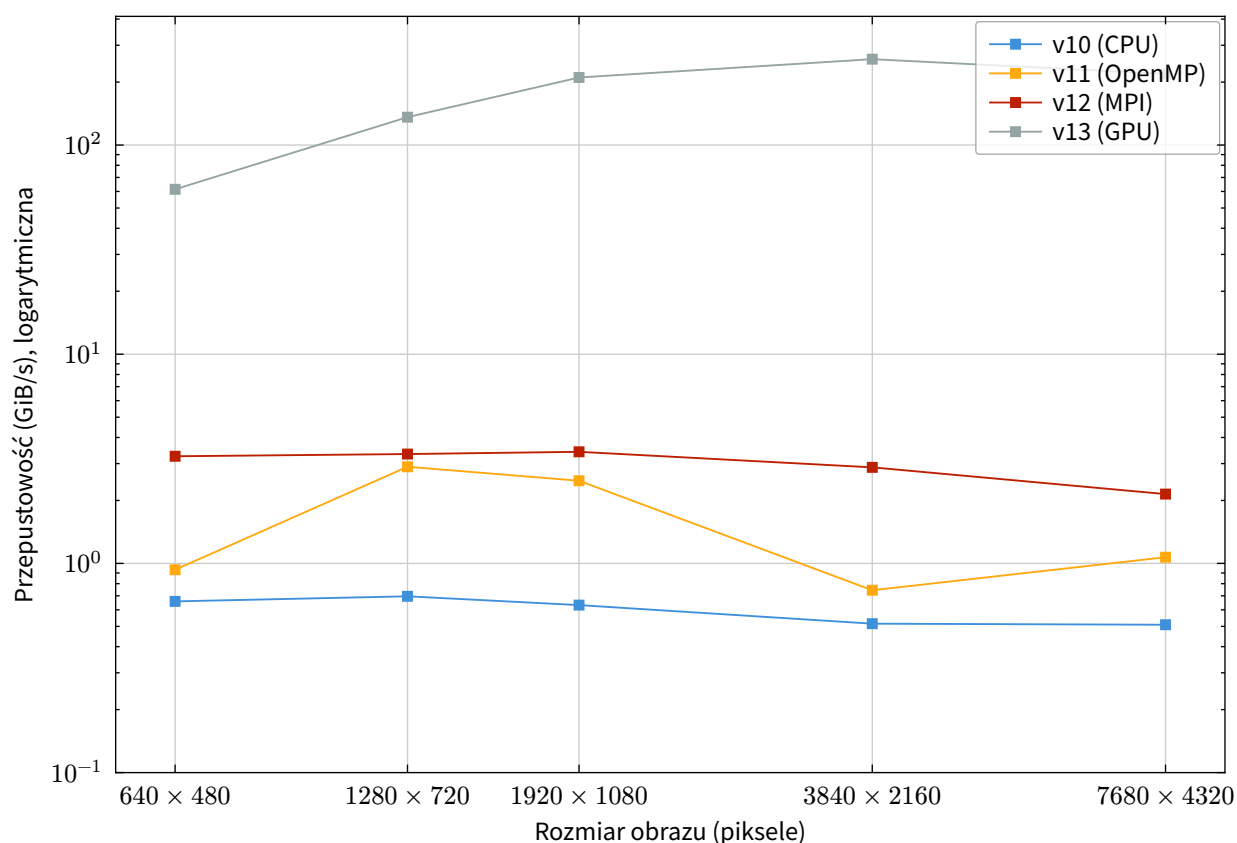
Wykorzystywany tu SYCL zapewnia dwie abstrakcje mające to ułatwić: bufory, które zapewniają prosty interfejs programistyczny, ale okazały się trudniejsze do optymalizacji, oraz Unified Shared Memory, zapewniającą dostęp wskaźnikowy do pamięci współdzielonej i możliwość manualnego zarządzania alokacją pamięci na urządzeniach. Koszt kopiowania pamięci na urządzenie jest znaczącym stałym kosztem i ma duży wpływ na wydajność rozwiązania.

Kolejnym aspektem wpływającym na wydajność jest projekt podziału algorytmu na wiele kerneli. Istotne jest dobranie odpowiedniego rozmiaru grup (powinny wykorzystywać wielokrotności rozmiaru warp/wavefront) i właściwe zarządzanie pamięcią w ramach grupy. Częstym podejściem do optymalizacji programów na GPU jest podział problemów na kafelki rdzeni z wydzieloną dla kafelek pamięcią współdzieloną, co pozwala grupować dostępy do pamięci. Zastosowane tu podejście do kafelkowania okazało się mało skuteczne, osiągając gorszą wydajność niż prostsze metody optymalizacji, nawet pomimo wielu prób. Prawdopodobnie możliwe byłoby zaprojektowanie lepszego kafelkowania, ale używane kafelki były zbyt małe, by zysk z przyspieszenia przeważał koszty synchronizacji pamięci między elementami kafelek.

Ostatecznie najlepsze wyniki osiągnęła wersja realizująca nieco więcej w ramach pojedynczego wątku — obliczając dwa piksele naraz (zwiększenie do czterech na wykorzystywanej karcie prowadziło do spadku wydajności) i tworząca grupy będące wielokrotnością rozmiaru wavefrontu. Osiągnięte

w ten sposób ponad 589GiB/s w najlepszym wypadku znacząco zbliża się do przepustowości pamięci w używanej karcie (624GiB/s).

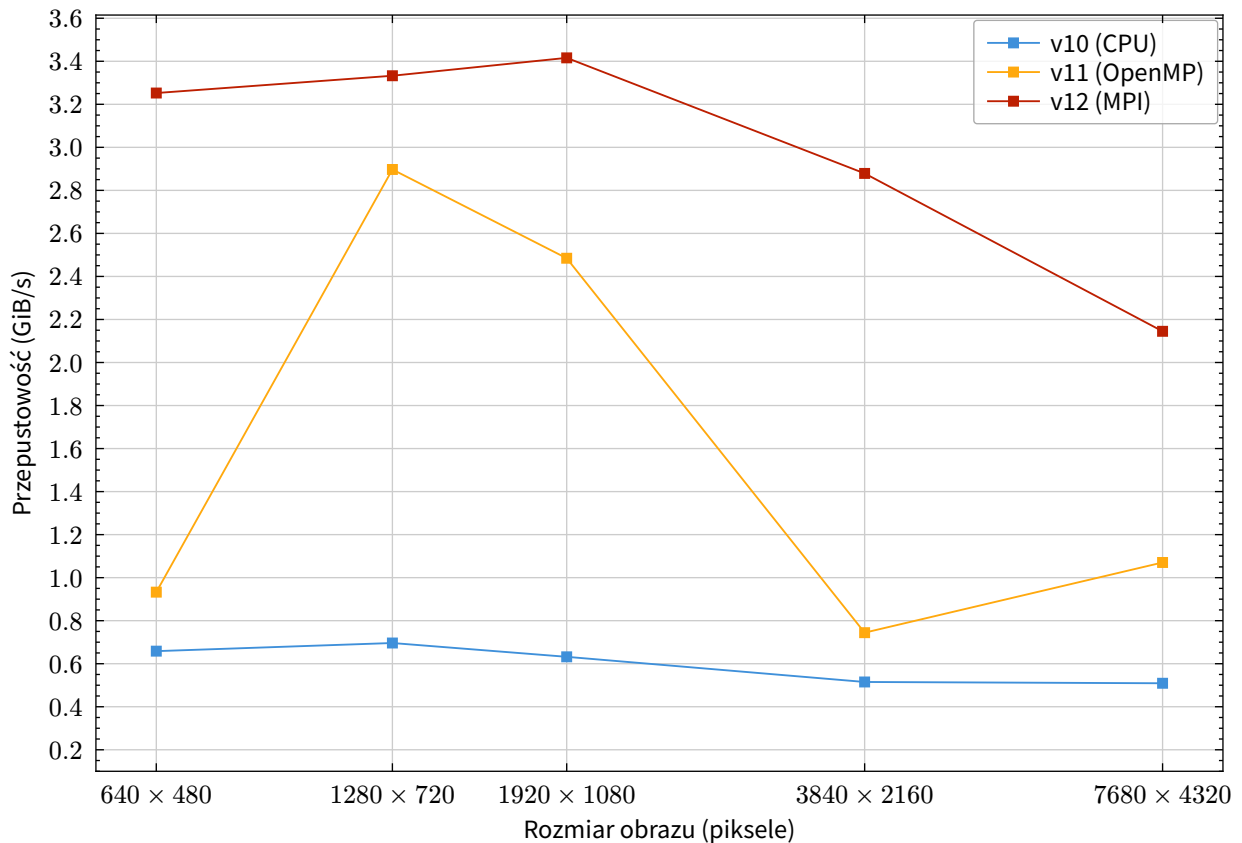
Takie prędkości dotyczą jedynie rdzenia Sobela. Dodając pozostałe etapy przetwarzania — transformację na skalę szarości i końcową normalizację wyników do zapisu — wydajność spada, pozostając jednak na poziomie ponad 300 razy wyższym niż oryginalna implementacja sekwencyjna.



Rysunek 4. Pełny pipeline przetwarzania obrazu

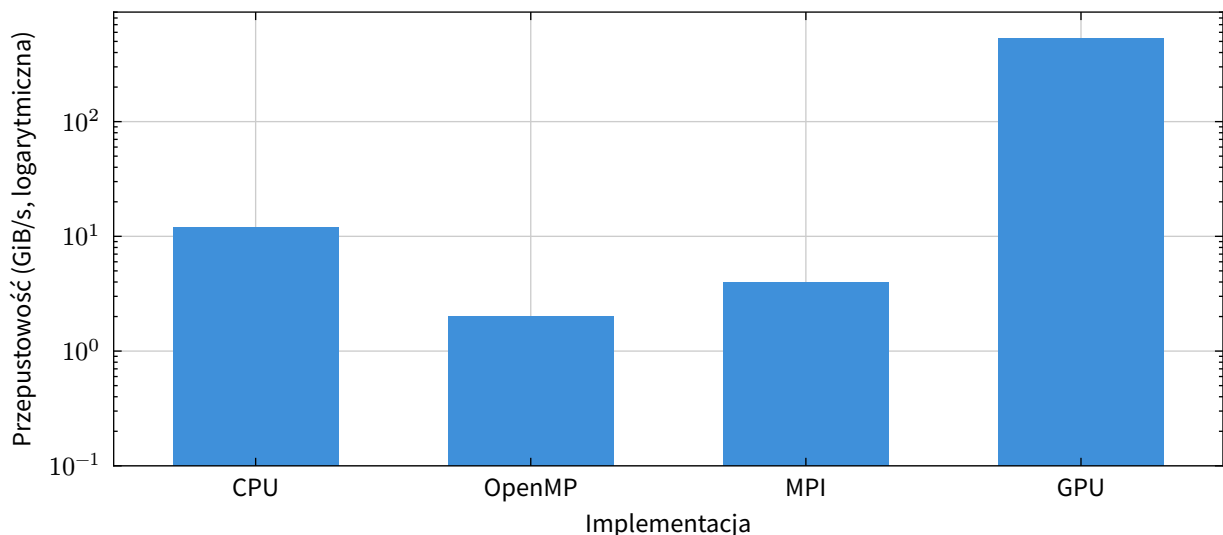
Patrząc na pełny program na [Rysunku 4](#) widzimy też, że relatywnie do GPU nasze rozwiązania wielowątkowe zapewniły znacznie mniejszy wzrost wydajności, jednak skala tutaj jest zakłamywana przez oś logarytmiczną i patrząc na [Rysunek 5](#) możemy zauważyć, że był to jednak znaczniejszy wzrost niż mogło nam się wydawać. To kilkukrotne przyspieszenie nie jest jednak wynikiem złym.

Co ciekawe, mimo narzutu wiadomości, wersja na MPI sprawdziła się tu znacząco lepiej niż OpenMP. Do tego wszędzie dla CPU widzimy szczyt wydajności w okolicy obrazów HD/Full-HD, dalej uzyskując coraz gorszą przepustowość.



Rysunek 5. Pełny pipeline przetwarzania obrazu (bez GPU)

Jednak wracając do samych kerneli Sobela, ciekawą sprawą jest też porównanie sekwencyjnej wersji zoptymalizowanej z resztą, widoczne na [Rysunku 6](#)



Rysunek 6. Najlepsze wyniki dla obrazu 7680 × 4320 pikseli

Jak widać, wydajność GPU dominuje - jednak sporym zaskoczeniem okazało się to, jak imponująco wyniki poprawił unrolling i optymalizacja wewnętrznej pętli algorytmu konwolucji, pozwalając dla największych obrazów sekwencyjnie pobić wyniki zrównoleglone na procesorze!

Oczywiście, sugeruje to, że zastosowanie tej samej optymalizacji dla wersji zrównoleglonych może pozwolić na osiągnięcie jeszcze lepszych wyników, ale jednak prowadzi do dość ważnego wniosku:

z wolnym algorytmem i kosztem wielkiego O wynikającego z wielu zagnieżdżonych pętli nie jest w stanie pomóc żadne rozwiązanie do rozpraszania obliczeń na wiele rdzeni.

5 Podsumowanie i wnioski

W ramach projektu zapoznaliśmy się i zaimplementowaliśmy operację splotu na obrazie z wykorzystaniem filtra do wykrywania krawędzi, bazując na filtrze Sobela.

Po stworzeniu implementacji sekwencyjnej zrównolegliliśmy działanie kodu na 3 różne sposoby, poznając przy okazji biblioteki SYCL, OpenMP i MPI. Przy każdej metodzie zaobserwowaliśmy znaczący wzrost wydajności jaki ten algorytm wykazuje przy zrównolegleniu.

Jednocześnie jednak mogliśmy zaobserwować, że nieraz znaczące zyski można uzyskać optymalizując sam algorytm - trochę pracy nad wersją sekwencyjną pozwoliło uzyskać na jednym rdzeniu wydajność lepszą niż naiwny algorytm był w stanie osiągnąć na 12.

Projekt dał nam więc okazję do zobaczenia jak znaczący stosunek zysku wydajności do włożonego wysiłku może nieraz zapewnić zrównoleglenie z użyciem współczesnych bibliotek i standardów, co najpewniej będziemy próbować aplikować w naszych przyszłych projektach programistycznych.

Bibliografia

- [1] AutomaticAddison, „How the Sobel Operator Works”. Dostęp: 8 grudnia 2025. [Online]. Dostępne na: <https://automaticaddison.com/how-the-sobel-operator-works/>
- [2] Wikimedia Foundation, „Sobel operator”. Dostęp: 8 grudnia 2025. [Online]. Dostępne na: https://en.wikipedia.org/wiki/Sobel_operator
- [3] Google, „benchmark: A microbenchmark support library”. [Online]. Dostępne na: <https://github.com/google/benchmark>

Spis rysunków

Rysunek 1	obraz w odcieniach szarości[2]	2
Rysunek 2	wynik użycia operatora Sobela[2]	2
Rysunek 3	Wszystkie wyniki benchmarków	9
Rysunek 4	Pełny pipeline przetwarzania obrazu	11
Rysunek 5	Pełny pipeline przetwarzania obrazu (bez GPU)	12
Rysunek 6	Najlepsze wyniki dla obrazu 7680 × 4320 pikseli	12

Spis tabel

Spis załączników

Załącznik A	Pełne wyniki benchmarków	14
-------------	--------------------------	----

Załącznik A Pełne wyniki benchmarków

ID	Nazwa	Rozmiar	Czas	Przepustowość (B/s)	Przepustowość (items/s)
v1	Sobel_CPU_Original	640 × 480	5.71 ms	412.1 MiB/s ± 11.15 MiB/s	54.01 M/s ± 1.46 M/s
v1	Sobel_CPU_Original	1280 × 720	17.21 ms	410.73 MiB/s ± 12.2 MiB/s	53.83 M/s ± 1.6 M/s
v1	Sobel_CPU_Original	1920 × 1080	41.51 ms	383.07 MiB/s ± 2.65 MiB/s	50.21 M/s ± 0.35 M/s
v1	Sobel_CPU_Original	3840 × 2160	168.11 ms	378.09 MiB/s ± 4.86 MiB/s	49.56 M/s ± 0.64 M/s
v1	Sobel_CPU_Original	7680 × 4320	717.92 ms	354.41 MiB/s ± 12.56 MiB/s	46.45 M/s ± 1.65 M/s
v2	Sobel_CPU_Unrolled	640 × 480	0.18 ms	12.63 GiB/s ± 0.08 GiB/s	1.7 G/s ± 0.01 G/s
v2	Sobel_CPU_Unrolled	1280 × 720	0.54 ms	12.79 GiB/s ± 0.11 GiB/s	1.72 G/s ± 0.01 G/s
v2	Sobel_CPU_Unrolled	1920 × 1080	1.23 ms	12.56 GiB/s ± 0.2 GiB/s	1.69 G/s ± 0.03 G/s
v2	Sobel_CPU_Unrolled	3840 × 2160	5.39 ms	11.52 GiB/s ± 0.18 GiB/s	1.55 G/s ± 0.02 G/s
v2	Sobel_CPU_Unrolled	7680 × 4320	23.14 ms	10.79 GiB/s ± 0.92 GiB/s	1.45 G/s ± 0.12 G/s
v3	Sobel_OpenMP	640 × 480	5.27 ms	549.44 MiB/s ± 68.2 MiB/s	64.01 M/s ± 7.95 M/s
v3	Sobel_OpenMP	1280 × 720	11.97 ms	1.28 GiB/s ± 0.13 GiB/s	153.25 M/s ± 15.09 M/s
v3	Sobel_OpenMP	1920 × 1080	27.59 ms	1.07 GiB/s ± 0.11 GiB/s	127.28 M/s ± 13.12 M/s
v3	Sobel_OpenMP	3840 × 2160	104.12 ms	1016.19 MiB/s ± 191.82 MiB/s	118.39 M/s ± 22.35 M/s
v3	Sobel_OpenMP	7680 × 4320	386.06 ms	889.63 MiB/s ± 136.71 MiB/s	103.65 M/s ± 15.93 M/s
v4	Sobel_MPI	640 × 480	0.35 ms	6.45 GiB/s ± 0.01 GiB/s	865.55 M/s ± 1.86 M/s
v4	Sobel_MPI	1280 × 720	1.02 ms	6.75 GiB/s ± 0.04 GiB/s	906.28 M/s ± 5.75 M/s
v4	Sobel_MPI	1920 × 1080	2.18 ms	7.09 GiB/s ± 0.03 GiB/s	951.07 M/s ± 4.59 M/s
v4	Sobel_MPI	3840 × 2160	12.68 ms	4.87 GiB/s ± 0.01 GiB/s	654.22 M/s ± 1.65 M/s
v4	Sobel_MPI	7680 × 4320	73.84 ms	3.35 GiB/s ± 0.01 GiB/s	449.32 M/s ± 1.07 M/s
v5	Sobel_Buffers_Naive	640 × 480	0.44 ms	12.57 GiB/s ± 0.48 GiB/s	1.69 G/s ± 0.06 G/s
v5	Sobel_Buffers_Naive	1280 × 720	0.69 ms	44.11 GiB/s ± 10.8 GiB/s	5.92 G/s ± 1.45 G/s
v5	Sobel_Buffers_Naive	1920 × 1080	1.13 ms	83.33 GiB/s ± 4.16 GiB/s	11.18 G/s ± 0.56 G/s
v5	Sobel_Buffers_Naive	3840 × 2160	3.54 ms	183.59 GiB/s ± 15.72 GiB/s	24.64 G/s ± 2.11 G/s
v5	Sobel_Buffers_Naive	7680 × 4320	13.54 ms	196.36 GiB/s ± 1.34 GiB/s	26.35 G/s ± 0.18 G/s
v6	Sobel_Buffers_Unrolled	640 × 480	0.27 ms	43.94 GiB/s ± 0.5 GiB/s	5.9 G/s ± 0.07 G/s
v6	Sobel_Buffers_Unrolled	1280 × 720	0.66 ms	108.08 GiB/s ± 0.66 GiB/s	14.51 G/s ± 0.09 G/s
v6	Sobel_Buffers_Unrolled	1920 × 1080	1.07 ms	206.6 GiB/s ± 1.39 GiB/s	27.73 G/s ± 0.19 G/s
v6	Sobel_Buffers_Unrolled	3840 × 2160	3.38 ms	416.15 GiB/s ± 1.32 GiB/s	55.85 G/s ± 0.18 G/s
v6	Sobel_Buffers_Unrolled	7680 × 4320	12.77 ms	500.28 GiB/s ± 3.48 GiB/s	67.15 G/s ± 0.47 G/s
v7	Sobel_DeviceUSM_Basic	640 × 480	0.03 ms	85.06 GiB/s ± 0.27 GiB/s	11.42 G/s ± 0.04 G/s
v7	Sobel_DeviceUSM_Basic	1280 × 720	0.04 ms	199.78 GiB/s ± 0.62 GiB/s	26.81 G/s ± 0.08 G/s
v7	Sobel_DeviceUSM_Basic	1920 × 1080	0.06 ms	326.04 GiB/s ± 1.11 GiB/s	43.76 G/s ± 0.15 G/s
v7	Sobel_DeviceUSM_Basic	3840 × 2160	0.14 ms	475.25 GiB/s ± 1.43 GiB/s	63.79 G/s ± 0.19 G/s
v7	Sobel_DeviceUSM_Basic	7680 × 4320	0.54 ms	465.37 GiB/s ± 0.9 GiB/s	62.46 G/s ± 0.12 G/s
v8	Sobel_DeviceUSM_Tiled_Demo	640 × 480	0.04 ms	82.52 GiB/s ± 0.23 GiB/s	11.08 G/s ± 0.03 G/s
v8	Sobel_DeviceUSM_Tiled_Demo	1280 × 720	0.04 ms	189.38 GiB/s ± 0.69 GiB/s	25.42 G/s ± 0.09 G/s
v8	Sobel_DeviceUSM_Tiled_Demo	1920 × 1080	0.06 ms	298.56 GiB/s ± 0.29 GiB/s	40.07 G/s ± 0.04 G/s

ID	Nazwa	Rozmiar	Czas	Przepustowość (B/s)	Przepustowość (items/s)
v8	Sobel_DeviceUSM_Tiled_Demo	3840 × 2160	0.15 ms	435.18 GiB/s ± 1.07 GiB/s	58.41 G/s ± 0.14 G/s
v8	Sobel_DeviceUSM_Tiled_Demo	7680 × 4320	0.68 ms	372.94 GiB/s ± 3.26 GiB/s	50.05 G/s ± 0.44 G/s
v9	Sobel_Optimized_Combo	640 × 480	0.04 ms	87.5 GiB/s ± 0.67 GiB/s	11.74 G/s ± 0.09 G/s
v9	Sobel_Optimized_Combo	1280 × 720	0.04 ms	214.51 GiB/s ± 0.58 GiB/s	28.79 G/s ± 0.08 G/s
v9	Sobel_Optimized_Combo	1920 × 1080	0.05 ms	359.58 GiB/s ± 0.91 GiB/s	48.26 G/s ± 0.12 G/s
v9	Sobel_Optimized_Combo	3840 × 2160	0.11 ms	588.24 GiB/s ± 1.93 GiB/s	78.95 G/s ± 0.26 G/s
v9	Sobel_Optimized_Combo	7680 × 4320	0.51 ms	493.09 GiB/s ± 1.87 GiB/s	66.18 G/s ± 0.25 G/s
v10	FullPipeline_CPU	640 × 480	5.64 ms	627.88 MiB/s ± 41.03 MiB/s	54.87 M/s ± 3.59 M/s
v10	FullPipeline_CPU	1280 × 720	15.95 ms	663.95 MiB/s ± 0.34 MiB/s	58.02 M/s ± 0.03 M/s
v10	FullPipeline_CPU	1920 × 1080	39.67 ms	602.75 MiB/s ± 41.26 MiB/s	52.67 M/s ± 3.61 M/s
v10	FullPipeline_CPU	3840 × 2160	194.12 ms	491.23 MiB/s ± 12.35 MiB/s	42.92 M/s ± 1.08 M/s
v10	FullPipeline_CPU	7680 × 4320	784.89 ms	485.53 MiB/s ± 9.54 MiB/s	42.43 M/s ± 0.83 M/s
v11	FullPipeline_OpenMP	640 × 480	4.16 ms	889.42 MiB/s ± 124.86 MiB/s	77.72 M/s ± 10.91 M/s
v11	FullPipeline_OpenMP	1280 × 720	8.63 ms	2.7 GiB/s ± 0.39 GiB/s	241.43 M/s ± 35.13 M/s
v11	FullPipeline_OpenMP	1920 × 1080	15.2 ms	2.31 GiB/s ± 0.56 GiB/s	207.05 M/s ± 49.84 M/s
v11	FullPipeline_OpenMP	3840 × 2160	145.48 ms	709.76 MiB/s ± 46.97 MiB/s	62.02 M/s ± 4.1 M/s
v11	FullPipeline_OpenMP	7680 × 4320	509.42 ms	1020.9 MiB/s ± 100.34 MiB/s	89.21 M/s ± 8.77 M/s
v12	FullPipeline_MPI	640 × 480	1.13 ms	3.03 GiB/s ± 0.02 GiB/s	271.04 M/s ± 1.62 M/s
v12	FullPipeline_MPI	1280 × 720	3.32 ms	3.1 GiB/s ± 0.01 GiB/s	277.73 M/s ± 1.22 M/s
v12	FullPipeline_MPI	1920 × 1080	7.28 ms	3.18 GiB/s ± 0 GiB/s	284.66 M/s ± 0.27 M/s
v12	FullPipeline_MPI	3840 × 2160	34.57 ms	2.68 GiB/s ± 0 GiB/s	239.9 M/s ± 0.42 M/s
v12	FullPipeline_MPI	7680 × 4320	185.62 ms	2 GiB/s ± 0.02 GiB/s	178.75 M/s ± 1.52 M/s
v13	FullPipeline_SYCL	640 × 480	0.08 ms	57.22 GiB/s ± 0.2 GiB/s	5.12 G/s ± 0.02 G/s
v13	FullPipeline_SYCL	1280 × 720	0.1 ms	126.59 GiB/s ± 1.23 GiB/s	11.33 G/s ± 0.11 G/s
v13	FullPipeline_SYCL	1920 × 1080	0.14 ms	195.92 GiB/s ± 0.77 GiB/s	17.53 G/s ± 0.07 G/s
v13	FullPipeline_SYCL	3840 × 2160	0.41 ms	239.69 GiB/s ± 1.52 GiB/s	21.45 G/s ± 0.14 G/s
v13	FullPipeline_SYCL	7680 × 4320	1.85 ms	203.78 GiB/s ± 1.12 GiB/s	18.23 G/s ± 0.1 G/s

Spis rysunków

Rysunek 1	obraz w odcieniach szarości[2]	2
Rysunek 2	wynik użycia operatora Sobela[2]	2
Rysunek 3	Wszystkie wyniki benchmarków	9
Rysunek 4	Pełny pipeline przetwarzania obrazu	11
Rysunek 5	Pełny pipeline przetwarzania obrazu (bez GPU)	12
Rysunek 6	Najlepsze wyniki dla obrazu 7680×4320 pikseli	12